

Modern Java (Java 11 → 21)

Complete Notes + Interview Q&A
Records • Pattern Matching for switch • Sealed Classes • Virtual Threads (Project Loom)

For a 4-Year Experienced Java Full Stack Developer

Interview mindset: Know the definition, the problem solved, how it works, code, real-world use, and trade-offs. These notes use simple language while covering the details expected in experienced Java interviews.

1. Modern Java 11 → 21

Java 11, 17 and 21 are important LTS releases. Java 21 is especially important for modern backend development because it combines cleaner data/domain modeling with major concurrency improvements.

The four topics in this guide are:

- Records — concise data carriers with generated boilerplate.
- Sealed classes — controlled inheritance and closed domain models.
- Pattern matching for switch — cleaner type-based decisions with less casting.
- Virtual threads — lightweight JVM-managed threads for high-concurrency workloads.

Simple mental model: Records simplify data. Sealed classes control types. Pattern matching simplifies decisions. Virtual threads simplify concurrency.

2. Records

A record is a special kind of class intended mainly to represent data. Java automatically provides the common boilerplate that a data class normally needs.

Records became permanent in Java 16, so they are available in Java 17 and 21.

Basic record

```
public record Employee(Long id, String name, String email) {}
```

For the components above, Java provides a canonical constructor, accessors `id()`, `name()`, `email()`, and implementations of `equals()`, `hashCode()`, and `toString()`.

Important points

- Record components are final references.
- Records are implicitly final and cannot be extended.
- A record can implement interfaces.
- A record cannot extend another class because it already extends `java.lang.Record`.
- Records are shallowly immutable: a final `List` reference does not make the `List` itself immutable.
- Record accessors are `name()`, `id()`, etc., not `getName()` or `getId()`.

Compact constructor and validation

```
public record Employee(Long id, String name) {  
    public Employee {  
        if (id == null || id <= 0)  
            throw new IllegalArgumentException("Invalid id");  
        if (name == null || name.isBlank())  
            throw new IllegalArgumentException("Invalid name");  
    }  
}
```

The compact constructor is useful for validation or normalization. Java handles assigning the validated parameters to the record components.

Defensive copy for mutable data

```
public record Team(String name, List<String> members) {  
    public Team {  
        members = List.copyOf(members);  
    }  
}
```

`List.copyOf` creates an unmodifiable copy, giving stronger immutability guarantees for the component.

Record vs normal class

Feature	Record	Normal class
Purpose	Data carrier	General object
State	Final components	Can be mutable
Inheritance	Cannot extend class	Can extend class
Boilerplate	Low	Usually higher
Accessor	name()	Often getName()
equals/hashCode	Generated	Developer/framework dependent

Records in Spring Boot

Records are a strong fit for request/response DTOs, projections and value-like objects. They are usually not the first choice for JPA entities because typical entities need mutable state, lifecycle behavior and proxy support.

```
public record EmployeeResponse(Long id, String name, String department) {}

@GetMapping("/employees/{id}")
public EmployeeResponse get(@PathVariable Long id) {
    Employee e = service.findById(id);
    return new EmployeeResponse(e.getId(), e.getName(), e.getDepartment());
}
```

3. Sealed Classes

A sealed class or interface restricts which types can directly extend or implement it. It is useful when the domain has a known set of valid subtypes.

```
public sealed interface Payment
    permits CardPayment, UpiPayment, CashPayment {}

public final class CardPayment implements Payment {}
public final class UpiPayment implements Payment {}
public non-sealed class CashPayment implements Payment {}
```

Every direct permitted subclass chooses one of three paths:

- final — inheritance stops here.
- sealed — inheritance remains restricted and another permits list is defined.
- non-sealed — inheritance is reopened for that branch.

Why use sealed classes?

- Make business/domain boundaries explicit.
- Prevent unwanted implementations.
- Make exhaustive pattern matching possible.
- Improve readability and maintainability.
- Combine naturally with records and pattern matching.

Example domain model

```
public sealed interface Result<T>
    permits Success, Failure {}

public record Success<T>(T value) implements Result<T> {}
public record Failure<T>(String message) implements Result<T> {}
```

A sealed type is about inheritance control, not immutability. A sealed class can still have mutable fields.

4. Pattern Matching for switch

Pattern matching for switch allows switch to test types and bind a variable at the same time. It reduces instance of chains and explicit casts. Pattern matching for switch was finalized in Java 21.

Before

```
if (value instanceof String) {
    String s = (String) value;
    return s.length();
}
```

Modern form

```
if (value instanceof String s) {
    return s.length();
}
```

Switch pattern example

```
static String describe(Object value) {
    return switch (value) {
        case String s -> "String: " + s;
        case Integer i -> "Integer: " + i;
        case null -> "null";
        default -> "Other";
    };
}
```

The pattern variable exists only where the match is definitely true.

Record patterns

```
record Point(int x, int y) {}

static String show(Object obj) {
    return switch (obj) {
        case Point(int x, int y) -> "Point: " + x + ", " + y;
        default -> "Other";
    };
}
```

Record patterns deconstruct a record directly, so you do not need to manually call x() and y() first.

Sealed classes + pattern switch

```
sealed interface Shape permits Circle, Rectangle {}

record Circle(double radius) implements Shape {}
record Rectangle(double width, double height) implements Shape {}

static double area(Shape s) {
    return switch (s) {
        case Circle c -> Math.PI * c.radius() * c.radius();
        case Rectangle r -> r.width() * r.height();
    };
}
```

Because Shape is sealed and all permitted direct types are covered, the switch can be exhaustive without default.

null and switch

A null selector historically caused NullPointerException in switch. Modern pattern switch lets you explicitly handle null with `case null`. Treat null deliberately rather than assuming it is covered by default.

Arrow cases and yield

- case X -> expression is concise and prevents accidental fall-through.
- case X -> { ... yield value; } is used when a switch-expression case needs a block.
- Java 21 should not be described as supporting a 'when' guard syntax; that belongs to later Java evolution.

```
static String classify(Integer n) {
    return switch (n) {
        case null -> "null";
        case Integer i -> {
            if (i > 0) yield "positive";
            if (i < 0) yield "negative";
            yield "zero";
        }
    };
}
```

5. Virtual Threads — Project Loom

Virtual threads were finalized in Java 21. They are lightweight `java.lang.Thread` instances managed by the JVM and are designed to make very large numbers of concurrent tasks practical.

The important idea

A platform thread is relatively expensive and maps closely to an operating-system thread. A virtual thread is scheduled by the JVM onto carrier platform threads. Many virtual threads can therefore share a much smaller set of carrier threads over time.

Creating one

```
Thread.startVirtualThread(() -> {
    System.out.println("Hello from a virtual thread");
});
```

Virtual-thread executor

```
try (ExecutorService executor =
    Executors.newVirtualThreadPerTaskExecutor()) {

    Future<String> a = executor.submit(() -> callServiceA());
    Future<String> b = executor.submit(() -> callServiceB());

    System.out.println(a.get());
    System.out.println(b.get());
}
```

Why it helps with I/O

Suppose a request calls a database or remote HTTP service and spends most of its time waiting. With traditional platform-thread-per-request designs, the expensive thread can sit idle during that wait. Virtual threads are designed so the JVM can park/suspend the virtual thread at suitable blocking points and let the carrier execute other runnable virtual threads.

Loom mechanics: mount and unmount

- A virtual thread executes on a carrier platform thread.
- When it reaches a blocking operation that can be handled by the runtime, the virtual thread can be unmounted/parked.
- The carrier is then available to run another virtual thread.
- When the blocked operation completes, the virtual thread becomes runnable again and can be mounted on a carrier.
- This is why the model scales much better for large numbers of mostly-waiting tasks.

Easy analogy: a carrier thread is like a waiter and virtual threads are customers. A customer waiting for food does not need the waiter to stand beside them doing nothing.

What virtual threads do NOT do

- They do not create more CPU cores.
- They do not make CPU-heavy algorithms automatically faster.
- They do not remove database connection limits.
- They do not remove HTTP rate limits or downstream capacity limits.

- They are not a reason to make every resource unbounded.

Production considerations

- Keep database pools bounded.
- Use timeouts, rate limits and sensible retries for downstream calls.
- Monitor memory, latency and queue/resource saturation.
- Review synchronized/native blocking sections where virtual-thread pinning can reduce scalability.
- Be careful with huge amounts of per-thread ThreadLocal state.

Structured concurrency note

Structured concurrency was a preview feature in the Java 21 era and continued evolving afterward. Do not call it a finalized Java 21 feature. Virtual threads are the major finalized Loom feature in Java 21.

6. Combining the Features

```
public sealed interface PaymentResult
permits Approved, Declined {}

public record Approved(String transactionId)
implements PaymentResult {}

public record Declined(String reason)
implements PaymentResult {}

static String message(PaymentResult r) {
return switch (r) {
case Approved a -> "Approved: " + a.transactionId();
case Declined d -> "Declined: " + d.reason();
};
}
```

This is a modern Java design: records model compact data, the sealed interface defines the allowed states, and pattern matching handles those states safely and exhaustively.

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
executor.submit(() -> {
PaymentResult result = paymentService.process();
return message(result);
});
}
```

The result is less boilerplate in both the domain model and concurrency code, while keeping the business cases explicit.

7. Interview Q&A — Records

1. What is a record?

Answer: A special class designed mainly for data. The compiler supplies the canonical constructor, component accessors, equals(), hashCode() and toString().

2. Are records immutable?

Answer: They are shallowly immutable. Components are final references, but referenced mutable objects can still change. Use defensive copies when required.

3. Can a record extend another class?

Answer: No. A record already extends java.lang.Record. It can implement interfaces.

4. Can a record be subclassed?

Answer: No. Records are implicitly final.

5. Can records contain methods?

Answer: Yes. They can have instance/static methods, static fields, constructors and validation logic.

6. What is a compact canonical constructor?

Answer: A constructor without an explicit parameter list/body assignment boilerplate, commonly used for validation or normalization.

7. Why are record accessors different?

Answer: For record Employee(String name), the accessor is name(). Records intentionally use component-based accessors rather than JavaBean getName().

8. Can records be JPA entities?

Answer: Usually they are a poor fit for normal JPA entities because entities commonly require mutable lifecycle state, constructors and proxy behavior. Records are better for DTOs/value-like data.

9. Why use records for DTOs?

Answer: They remove boilerplate and communicate that the object is primarily a data carrier.

10. When should you avoid records?

Answer: When the object needs mutable identity-driven state, inheritance, special bean conventions, or framework behavior that does not fit record semantics.

8. Interview Q&A — Sealed Classes

11. What is a sealed class?

Answer: A class/interface that explicitly restricts which types may directly extend or implement it.

12. What is permits?

Answer: The permits clause lists the types allowed to directly extend/implement a sealed type when the compiler cannot infer the permitted types.

13. Explain final, sealed and non-sealed.

Answer: final stops inheritance; sealed continues a controlled hierarchy; non-sealed reopens inheritance.

14. Can an interface be sealed?

Answer: Yes.

15. Does sealed mean immutable?

Answer: No. It controls inheritance only.

16. How do sealed classes help switch?

Answer: The compiler knows the permitted type set, enabling exhaustive pattern matching.

17. Can a permitted subclass be sealed?

Answer: Yes. It can define its own permitted subclasses.

18. Why is this useful in business code?

Answer: It expresses a closed domain model such as PaymentResult, CommandResult, or known workflow states directly in the type system.

9. Interview Q&A — Pattern Matching

19. What problem does pattern matching for switch solve?

Answer: It combines type testing and variable binding, reducing instanceof chains and explicit casts.

20. Was pattern matching for switch finalized in Java 21?

Answer: Yes.

21. What is exhaustiveness?

Answer: A switch is exhaustive when all possible input categories are handled.

22. Why can a sealed-type switch omit default?

Answer: Because the compiler knows the permitted direct subtypes and can verify that all are covered.

23. How do you handle null?

Answer: Use case null when null needs explicit handling.

24. What are record patterns?

Answer: Patterns that deconstruct record components directly.

25. Why prefer arrow cases?

Answer: They avoid accidental fall-through and are clearer for switch expressions.

26. What does yield do?

Answer: It returns a value from a block used as one case of a switch expression.

10. Interview Q&A — Virtual Threads

27. What is a virtual thread?

Answer: A lightweight JVM-managed `java.lang.Thread` designed for very high concurrency. It was finalized in Java 21.

28. Virtual thread vs platform thread?

Answer: Platform threads are relatively expensive and tied closely to OS threads. Virtual threads are scheduled by the JVM over carrier platform threads.

29. Are virtual threads faster?

Answer: Not inherently. Their main advantage is scalability for large numbers of concurrent tasks, especially I/O-bound tasks.

30. What is a carrier thread?

Answer: A platform thread on which a virtual thread executes.

31. Explain mount/unmount.

Answer: A virtual thread is mounted on a carrier while executing. At suitable blocking points it can be parked/unmounted, freeing the carrier. It later becomes runnable and can be mounted again.

32. Best use cases?

Answer: High-concurrency, mostly I/O-bound workloads such as HTTP calls, database operations, file I/O and request processing.

33. Are they good for CPU-heavy work?

Answer: They can execute CPU work, but they do not add CPU capacity. CPU-heavy tasks still need bounded parallelism and appropriate CPU resources.

34. Should every request create a DB connection?

Answer: No. Use a bounded connection pool. Virtual threads make waiting for a connection cheaper; they do not increase database capacity.

35. What does `newVirtualThreadPerTaskExecutor()` do?

Answer: It creates an `ExecutorService` that starts a new virtual thread for each submitted task.

36. What is the biggest adoption mistake?

Answer: Treating cheap threads as unlimited system capacity. Downstream services, DB pools, CPU, memory and rate limits remain bounded.

37. What is thread pinning?

Answer: A situation where a virtual thread cannot efficiently unmount from its carrier during certain blocking sections, reducing scalability. Synchronized/native blocking sections deserve attention.

38. Can `ThreadLocal` be used?

Answer: Yes, but large-scale per-thread state should be reviewed carefully because there can be very large numbers of virtual threads.

39. Do virtual threads replace `CompletableFuture`?

Answer: No. They are different models. Virtual threads simplify thread-per-task blocking code; `CompletableFuture` is useful for asynchronous composition and stage-based APIs.

40. Are virtual threads daemon threads?

Answer: Yes. Virtual threads are daemon threads.

11. Scenario-Based Questions for a 4-Year Developer

41. A Spring Boot API calls three downstream services. How can virtual threads help?

Answer: If the calls are blocking I/O, virtual threads allow many requests to wait without consuming one expensive platform thread per blocked request. I would still use timeouts, bounded connection pools, retries, rate limits and bulkheads.

42. *Payment has Approved, Declined and Pending states. Would you use sealed types?*

Answer: Yes, if the set is controlled by the application. A sealed interface plus records creates a clear closed model, and a pattern switch can enforce handling of every state.

43. *Would you use records for every DTO?*

Answer: No. Use them where value-like immutable data is appropriate. Use a normal class when mutability, inheritance, bean conventions or framework requirements make it a better fit.

44. *Is default always good in a switch over a sealed type?*

Answer: No. If the domain is intentionally closed, omitting default can let the compiler reveal missing cases when the hierarchy evolves.

45. *With 100,000 concurrent requests, should you create 100,000 DB connections?*

Answer: No. Virtual threads are cheap tasks, not unlimited resources. Keep a bounded DB pool and let virtual threads wait efficiently for available connections.

46. *What is the overall benefit of modern Java?*

Answer: It reduces accidental complexity: records reduce data boilerplate, sealed types make domain boundaries explicit, pattern matching reduces casting and branching complexity, and virtual threads make high-concurrency blocking code simpler.

12. Quick Revision Sheet

Topic	Must remember
Records	Data carrier; final components; generated methods; shallow immutability; no class inheritance.
Sealed	Controlled inheritance; permits; final/sealed/non-sealed; strong with exhaustive switch.
Pattern switch	Type patterns + switch; finalized Java 21; case null; record patterns.
Virtual threads	Finalized Java 21; JVM-managed; carrier threads; best for high-concurrency I/O.
Loom mechanics	Mount on carrier → suitable blocking/park → unmount → resume later.
Production rule	Cheap virtual threads do not make DBs, CPUs or downstream APIs unlimited.

13. Interview Answer Formula

Definition → Problem → How it works → Code → Real-world use → Trade-off.

For experienced interviews, avoid saying only “this is a new Java feature.” Explain why you would choose it and what constraint still remains.

Example virtual-thread answer: “Virtual threads are lightweight JVM-managed threads finalized in Java 21. They mainly improve concurrency for blocking I/O. The JVM schedules them over carrier platform threads and can park them at suitable blocking points. I would use them for high-concurrency backend requests while keeping DB pools, downstream limits and timeouts bounded. They improve concurrency, not CPU capacity.”

14. Final Checklist

- Records: components, accessors, canonical/compact constructors, shallow immutability, DTO use.
- Sealed types: permits, final/sealed/non-sealed, exhaustive switch.
- Pattern matching: type patterns, switch expressions, case null, record patterns.
- Virtual threads: platform vs virtual, carrier threads, mount/unmount, I/O vs CPU, pinning and resource limits.

- Be able to combine all four features in a Spring Boot/backend design.
- Always discuss trade-offs and production limits.